Computational Economics

Lecture: Policy Function Iteration (PFI)

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lecture Overview

- **Policy Evaluation**
  - The policy evaluation operator
  - Repeated updates and a linear system
- **Policy Function Iteration**
  - Evaluation and improvement in the maze
  - The algorithm and its optimality criterion
- **Computational Choices**
  - Generalized and modified policy iteration
  - Choosing the amount of policy evaluation

From Value Iteration to Policy Iteration

Given the current value estimate V_k, a VFI update separates into two steps:

1. **Policy improvement**: choose a policy greedy with respect to V_k:

$$\sigma_{k+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V_k(s') \mid s, a]\}.$$

2. **Policy evaluation update**: hold σ_{k+1} fixed and update the value once:

$$V_{k+1}(s) = r(s, \sigma_{k+1}(s)) + \beta \mathbb{E}[V_k(s') \mid s, a = \sigma_{k+1}(s)].$$

Because the selected action is a maximizer, these steps give exactly the VFI update.

Policy evaluation computes a fixed policy's value. VFI takes only one update; generally, $V_{k+1} \neq V^{\sigma_{k+1}}$.

Policy function iteration (PFI) instead completes policy evaluation to obtain $V^{\sigma_{k+1}}$ before the next policy improvement.

The Policy Evaluation Operator

For a fixed policy σ, the **policy evaluation operator** T_σ maps a candidate continuation-value function V to an updated function:

$$(T_\sigma V)(s) = r(s, \sigma(s)) + \beta \mathbb{E}[V(s') \mid s, a = \sigma(s)].$$

- The action is supplied by $\sigma(s)$; this update does not choose a new action.
- The input V may be any bounded candidate function. It need not equal V^σ.

Policy evaluation means finding the fixed point:

$$V^\sigma = T_\sigma V^\sigma.$$

One application $T_\sigma V$ is one update; repeated updates compute the policy's value.

Iterative Policy Evaluation

Hold σ fixed. Choose V_0, and update all states using the same previous function:

$$V_{j+1}(s) = (T_\sigma V_j)(s) = r(s, \sigma(s)) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, \sigma(s)) V_j(s').$$

The index j counts evaluation updates, not changes of policy.

As with the Bellman optimality operator, discounting gives a contraction:

$$\|T_\sigma V_1 - T_\sigma V_2\|_\infty \leq \beta \|V_1 - V_2\|_\infty.$$

Hence $V_j \to V^\sigma$. For a target evaluation error $\varepsilon_{\text{eval}} > 0$, compute

$$d_j = \|V_{j+1} - V_j\|_\infty, \qquad \|V_j - V^\sigma\|_\infty \leq \frac{d_j}{1 - \beta}.$$

Stop and return V_j when $d_j \leq (1 - \beta)\varepsilon_{\text{eval}}$.

Policy Evaluation as a Linear System

For finitely many states, collect the rewards and transition probabilities selected by σ:

$$r^\sigma(s) = r(s, \sigma(s)), \qquad P^\sigma(s, s') = P(s' \mid s, \sigma(s)).$$

Each row of P^σ describes the next-state probabilities under the policy's chosen action.

The policy's Bellman equation becomes

$$V^\sigma = r^\sigma + \beta P^\sigma V^\sigma \qquad \Longleftrightarrow \qquad (I - \beta P^\sigma) V^\sigma = r^\sigma.$$

- I is the identity matrix. There is one equation and one unknown value per state.
- Solve this linear system directly, or use repeated evaluation updates.
- A direct solve avoids an inner iteration; its cost grows with the number of states.

Example: Evaluating the Maze Policy

Use the same policy σ as before, now with $\beta = 0.9$.

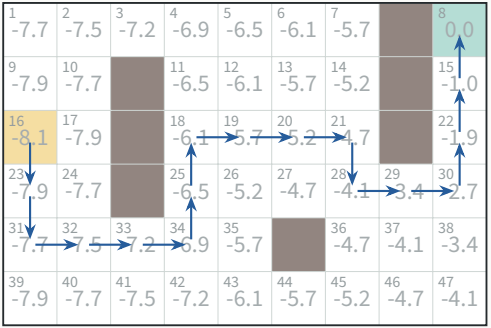


Figure 1: Discounted values under σ.

- Each move gives -1, including entry into cell 8; rewards afterward are zero.
- At cell 25, the policy chooses U to cell 18.

$$V^\sigma(8) = 0,$$
$$V^\sigma(25) = -1 + 0.9V^\sigma(18)$$
$$\approx -6.513.$$

At the starting cell, $V^\sigma(16) \approx -8.147$.

Cell values are rounded to one decimal place.

Example: Improving and Re-evaluating the Policy

Change only cell 25 to R: $\widetilde{\sigma}(25) = \mathrm{R}$; keep $\widetilde{\sigma}(s) = \sigma(s)$ elsewhere.

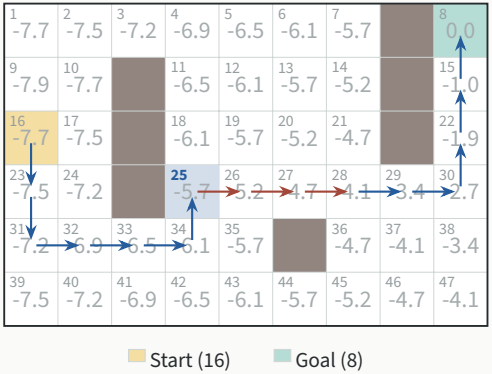


Figure 2: Re-evaluated values under $\widetilde{\sigma}$.

Policy improvement using V^{σ}:

$$Q^{\sigma}(25, \mathrm{U}) \approx -6.513,$$
$$Q^{\sigma}(25, \mathrm{R}) \approx -5.695.$$

Policy evaluation for the new policy:

$$V^{\widetilde{\sigma}}(25) \approx -5.695,$$
$$V^{\widetilde{\sigma}}(16) \approx -7.712.$$

The shortcut also improves values at states whose paths pass through cell 25.

Policy Function Iteration (PFI) Algorithm

0. **Input.** Finite state set $\mathcal{S}$, finite nonempty feasible action sets $\mathcal{A}(s)$, rewards $r(s,a)$, transition probabilities $P(s' \mid s,a)$, and discount factor β.
1. **Initialize.** Choose a feasible policy $\sigma_0(s) \in \mathcal{A}(s)$ for every state. Set $k = 0$.
2. **Policy evaluation.** Form r^{σ_k} and P^{σ_k}, then solve

$$(I - \beta P^{\sigma_k}) V^{\sigma_k} = r^{\sigma_k}.$$

3. **Compute action values.** For every feasible (s,a), compute

$$Q^{\sigma_k}(s,a) = r(s,a) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a) V^{\sigma_k}(s').$$

   Use this same evaluated V^{σ_k} for every action comparison.

PFI Algorithm (continued)

4. **Policy improvement.** For every state, select

$$\sigma_{k+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^{\sigma_k}(s, a).$$

   Keep $\sigma_k(s)$ if it is already a maximizer; otherwise use a fixed action order to break ties.

5. **Check the policy.**
   - If $\sigma_{k+1}(s) = \sigma_k(s)$ at every state, continue to **Step 6**.
   - Otherwise, set $k \leftarrow k + 1$ and return to **Step 2**.

6. **Return and stop.** Output the policy and its evaluated value:

$$\sigma^* = \sigma_k, \qquad V^* = V^{\sigma_k}.$$

The exact optimality statement uses exact policy evaluation. A numerical solve introduces rounding error; truncated evaluation introduces an additional approximation.

Policy Improvement and Optimality

Every greedy improvement satisfies the policy improvement theorem:

$$Q^{\sigma_k}(s, \sigma_{k+1}(s)) \geq V^{\sigma_k}(s) \quad \forall s \quad \Longrightarrow \quad V^{\sigma_{k+1}}(s) \geq V^{\sigma_k}(s) \quad \forall s.$$

When the policy does not change, it is greedy with respect to its own value:

$$V^{\sigma_k}(s) = \max_{a \in \mathcal{A}(s)} Q^{\sigma_k}(s, a) = (TV^{\sigma_k})(s).$$

Thus $V^{\sigma_k} = V^*$, by uniqueness of the Bellman optimality fixed point.

- Retaining the current action in a tie prevents changes between equally good actions.
- If the policy changes, its value strictly increases at some state.
- With finitely many deterministic policies, exact PFI therefore terminates at an optimal policy after finitely many improvements.

Modified Policy Iteration (MPI)

Modified policy iteration (MPI) performs $k \geq 1$ policy evaluation updates between policy improvements. Let i index the improvement rounds.

1. **Policy improvement.** Choose a policy greedy with respect to the current candidate V_i:

$$\sigma_{i+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V_i(s') \mid s, a]\}.$$

2. **Policy evaluation.** Hold σ_{i+1} fixed. Set $V_{i,0} = V_i$, then compute

$$V_{i,j+1} = T_{\sigma_{i+1}} V_{i,j}, \qquad j = 0, \ldots, k-1.$$

3. Set $V_{i+1} = V_{i,k}$ and repeat.

V_i is an approximate value, so an unchanged greedy policy alone does not certify optimality. Check the Bellman residual $\|TV_i - V_i\|_\infty$, as in VFI.

Generalized Policy Iteration

MPI illustrates **generalized policy iteration (GPI)**: alternate policy evaluation and policy improvement, without requiring evaluation to finish before improving the policy.

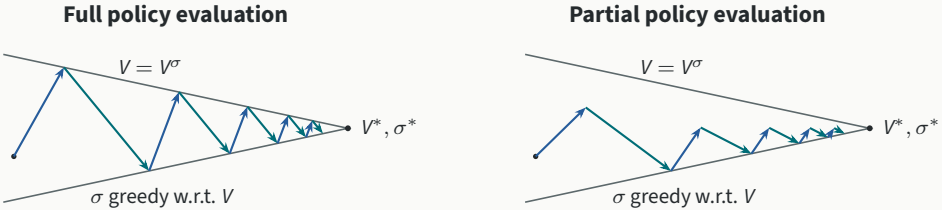


Figure 3: Full and partial policy evaluation within GPI (schematic).

Policy evaluation moves toward $V = V^\sigma$. **Policy improvement** makes σ greedy with respect to V.

At the intersection, both conditions hold: $V = T_\sigma V = TV$, so $V = V^*$ and σ is optimal.

Concept adapted from Sutton and Barto (2018), Section 4.6, pp. 86–87.

Choosing the Amount of Policy Evaluation

One policy evaluation update: VFI. Since σ_{i+1} is greedy with respect to V_i,

$$T_{\sigma_{i+1}} V_i = TV_i.$$

Setting $k = 1$ therefore gives $V_{i+1} = TV_i$.

Several policy evaluation updates: modified policy iteration. Extra updates reuse the chosen actions. They avoid a new maximization while refining the value estimate.

Full policy evaluation: PFI. Continuing evaluation to its fixed point gives $V^{\sigma_{i+1}}$; the next policy improvement uses that policy's value.

For a finite discounted MDP with bounded rewards, these exact updates converge to V^*, including any fixed $k \geq 1$.

The choice balances the cost of policy evaluation against the number of policy improvements.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Chapter 4: Dynamic Programming
2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming — Policy Function Iteration
   - Discrete State Dynamic Programming — Modified Policy Function Iteration